



Mission: The Smart Dice

Objective: To transform your `micro:bit` into a smart, interactive digital dice that can be activated by a gesture, and provide visual and audio feedback.

Tools & Resources:

- Your `micro:bit` device.
 - The `micro:bit` Python Editor: <https://python.microbit.org/>
 - The `micro:bit` Python Reference: <https://python.microbit.org/v/3/reference> (Your primary source of answers!)
-

Phase 1: The Basic Dice (Review and Launch)

You have been provided with the starting code for a basic dice.

Concept Check:

1. **Input:** What must you do to the `micro:bit`'s code for it to start running?
 2. **Output:** What kind of output does this initial code produce? What specific hardware component is responsible for showing the number?
 3. **Core Components:**
 - What does the `import random` line allow the code to do?
 - What is the role of the `while True:` loop in this code? (Hint: Why does the code repeat?)
-

Phase 2: Activating the Dice with a Gesture (Input Sensor Challenge)

The basic dice is annoying because it keeps rolling on its own! A real dice only rolls when you pick it up and shake it. We want the `micro:bit` to only roll when it senses a **shake**.

Mission Goal 2: Modify the code so the dice only rolls a new number when the `micro:bit` is shaken.

Conceptual Clues:

1. **New Input:** You need to use one of the **on-board sensors** to detect movement. Which part of the `micro:bit` is responsible for sensing motion? (Hint: Check the reference for sensors that detect acceleration or movement.)
2. **Detection:** How do you check if a specific **gesture** (like 'shake') has just occurred?
3. **Control Structure:** You will need an `if` statement to check for the gesture. The code to roll the dice (the `random.randint` and `display.show`) should only run *if* the shake is detected.

 **Task:** Use the `micro:bit` reference to find the tools that let you detect the **'shake' gesture** and implement this logic in your code.

Phase 3: Celebrate the Roll (Audio Output Challenge)

A good dice roll deserves some fanfare! Before the number is shown, we want the `micro:bit` to play a celebratory tune.

Mission Goal 3: Make the `micro:bit` play a short sound/jingle immediately before the new dice number is displayed.

Conceptual Clues:

1. **New Tool:** Playing music requires a specific **module** to be imported. What is the name of the module that handles sound and music? (Hint: Look for sections about music or sound in the reference.)
2. **Playing a Tune:** You need to find the specific command that tells the `micro:bit` to play a sequence of notes.
3. **Speaker:** What hardware component on the `micro:bit` is used to output the sound?

 **Task:** Use the `micro:bit` reference to find how to **import the music module** and then find a way to **play a note sequence** inside your `if accelerometer.was_gesture('shake'): block`.

Phase 4: Speak the Score (Conditional Audio Output Challenge)

Now that the dice is shaken, and a number is displayed, let's make the `micro:bit` announce the result!

Mission Goal 4: Make the `micro:bit` say the number it rolled after it is displayed. Create different spoken announcements for different numbers (e.g., 'it is onee!', 'threeeeeee', 'six, seven?').

Conceptual Clues:

1. **Another New Tool:** Saying words requires a different **module** than playing music. What is the name of the module that handles **speech synthesis**?
2. **Spoken Output:** What is the command that allows you to make the `micro:bit` speak a word or phrase?
3. **Making Decisions:** To say a *different* phrase for each number, you must use a set of **conditional statements** (`if`, `elif`). You will need to check the value of the variable holding the random number (`x`) and execute the correct speak command.

 **Task:** Find the **speech module** commands and use `if/elif` statements to make your `micro:bit` announce the rolled number with personality!

Phase 5: The Button Backup (Alternative Input Challenge)

Sometimes, you don't want to shake the device. Give the user an alternative way to roll the dice.

Mission Goal 5: Allow the user to roll the dice (triggering the music, display, and speech) by pressing either Button A or Button B, in addition to shaking it.

Conceptual Clues:

1. **Alternative Input:** Which part of the `micro:bit` is responsible for detecting a **button press**?
2. **Detecting a Press:** Find the command that checks if a specific button (like `button_a`) has been **pressed**.
3. **Combining Conditions:** You need your core dice-rolling code to run if **EITHER** the shake gesture is detected **OR** a button is pressed. This will require you to combine two separate conditions using a logical operator (`and`, `or`).

 **Task:** Find the **button-checking** commands and rewrite your main `if` statement to check for **either a shake OR a button press** to roll the dice.